

Hopfield model (Part 2)

Instructor: Mark Kramer

Today

Simulate the Hopfield model

Model system

Consider neuron i Use $S = 2V - 1$

$$S_i = \begin{cases} +1 & \text{if neuron is "firing at maximum rate"} \\ -1 & \text{if neuron is "not firing"} \end{cases}$$

Dynamics

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\} \quad \text{where we choose at random a neuron } i \text{ to update at each moment}$$

Model system

Consider neuron i Use $S = 2V - 1$

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\}$$

T_{ij} = connection to neuron i from neuron j

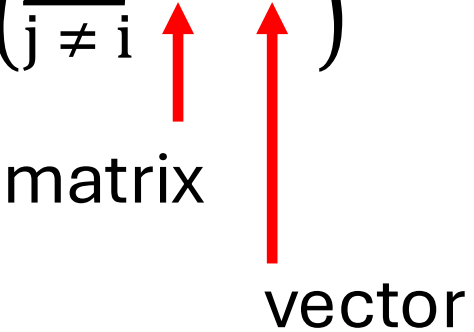
= 0 if not connected

Note: reciprocal connections are allowed

Q: What is T_{ij} ? To store pattern ξ : use $T_{ij} = \xi_i \xi_j$ where $T_{ii} = 0$

Matrix

We need to compute

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\}$$


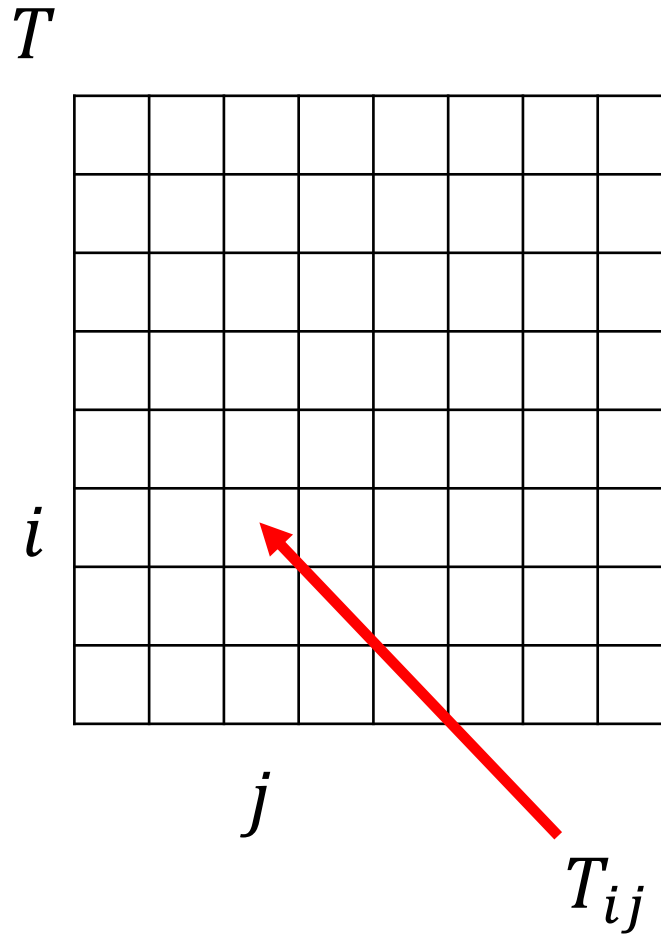
matrix

vector

More linear algebra ...

Matrix

A rectangular array of numbers



Matrix

A rectangular array of numbers

$$W = \begin{pmatrix} w_{11} & \cdots & w_{1m} \\ \vdots & & \vdots \\ w_{n1} & \cdots & w_{nm} \end{pmatrix} \quad n \times m \text{ matrix}$$

in Python

```
# make a 3 x 4 matrix  
W = np.array([[1, 7, 3, 0], [2, -1, 2, -1], [1, 1, 1, 1]])
```

Matrix

A rectangular array of numbers

$$W = \begin{pmatrix} w_{11} & \cdots & w_{1m} \\ \vdots & & \vdots \\ w_{n1} & \cdots & w_{nm} \end{pmatrix}$$

$n \times m$ matrix

can think of it as:

m column vectors

$$\begin{pmatrix} | & & | \\ \mathbf{c}_1 & \cdots & \mathbf{c}_m \\ | & & | \end{pmatrix}$$

n row vectors

or

$$\begin{pmatrix} \text{---} \mathbf{r}_1 \text{---} \\ \vdots \\ \text{---} \mathbf{r}_n \text{---} \end{pmatrix}$$

Matrix

A matrix can be

“tall and skinny”

rows > # columns

$$m \begin{bmatrix} n \\ A \end{bmatrix}$$

“short and fat”

rows < # columns

$$m \begin{bmatrix} n \\ A \end{bmatrix}$$

Matrix multiplication

$$\vec{u} = W \vec{v}$$

One perspective: dot product with each row:

$$\begin{array}{c} \mathbf{u} \\ i^{th} \text{ component} \end{array} \left[\bigcirc \right] = \begin{array}{c} \mathbf{W} \\ i^{th} \text{ row} \end{array} \left[\text{---} \right] \begin{array}{c} \mathbf{v} \\ \text{---} \end{array}$$

Matrix-vector multiplication

another perspective: *linear combination of columns*

$$\begin{array}{c} \vec{\mathbf{u}} \\ \left[\begin{array}{c} u_1 \\ \vdots \\ u_n \end{array} \right] \end{array} = \begin{array}{c} \mathbf{W} \\ \left[\begin{array}{c|c|c} \begin{array}{c} \downarrow \\ \vec{\mathbf{c}}_1 \\ \uparrow \end{array} & \dots & \begin{array}{c} \downarrow \\ \vec{\mathbf{c}}_m \\ \uparrow \end{array} \end{array} \right] \end{array} \begin{array}{c} \vec{\mathbf{v}} \\ \left[\begin{array}{c} v_1 \\ \vdots \\ v_m \end{array} \right] \end{array} = \begin{array}{c} \begin{array}{c} \downarrow \\ v_1 \cdot \vec{\mathbf{c}}_1 \\ \uparrow \end{array} + \begin{array}{c} \downarrow \\ v_2 \cdot \vec{\mathbf{c}}_2 \\ \uparrow \end{array} + \dots + \begin{array}{c} \downarrow \\ v_m \cdot \vec{\mathbf{c}}_m \\ \uparrow \end{array} \end{array}$$

Matrix-vector multiplication (by hand)

$$1) \begin{bmatrix} 5 & 3 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$2) \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 11 \end{bmatrix}$$

$$3) \begin{bmatrix} -2 & 6 \\ 1 & -3 \\ -5 & 15 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$4) \begin{bmatrix} 1 & 3 & 7 \\ 2 & 4 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 7 \end{bmatrix}$$

$$5) \begin{bmatrix} 3 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

diagonal matrix

$$6) \begin{bmatrix} 3 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Q1: What do you notice about the relationship between the size of the matrix and the size of the vector?

Q2: what does multiplying by a diagonal matrix do to a vector?

Matrix-vector multiplication

you will never (or rarely) need to do this by hand!

in python:

```
# make a 3 x 4 matrix
W = np.array([[1, 7, 3, 0], [2, -1, 2, -1], [1, 1, 1, 1]])

# make a 4 x 1 matrix (ie, a vector)
v = np.array([[1], [2], [-3], [0]])

# Compute W times v (matrix-vector product)
u = W @ v
```


note special symbol '@'
for matrix multiply!

Q: what size is u?

Information storage

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\}$$

Store the patterns in T_{ij}

To store pattern ξ : use $T_{ij} = \xi_i \xi_j$ where $T_{ii} = 0$

“The stored state would always be stable under our processing algorithm”

Let's try it.

Python

Linear Algebra: Matrices

Hopfield: Find the pattern